

Assignment 1 (Part A) — 32 Marks

Build a real-time chat application using .NET for Windows

Objective

You will build a **real-time chat application** for Windows. A user signs in with a user ID of their own choosing and sees the channels that currently exist. They can join one of those channels or create a channel of their own. Inside a channel they take part in the conversation with everyone else who is there, hold private conversations with individual members alongside it, and share files with the channel.

You will build this using C#, the .NET Framework, Windows Communication Foundation (WCF) and Windows Presentation Foundation (WPF). This is a group assignment for two or three members, though you may also work individually.

Your solution must contain at least three projects:

- **Chat Server** — a self-hosted WCF service, for which a console application is sufficient. It holds all shared state: who is signed in, which channels exist, who is in each one, and the files that have been shared.
- **Polling Client** — a WPF application providing the user interface described in Section A. It keeps itself up to date by asking the server for changes on a background thread. Several instances run at the same time.
- **Duplex Client** — a second WPF application offering the same functionality, in which the server pushes changes to the client instead. This is what Section C assesses.

Both clients connect to the same running server and share its state, so a user on the polling client and a user on the duplex client can be in the same channel and talk to each other. You are expected to share code between the two clients — the service contracts, the data classes and the windows themselves — rather than duplicating them, and placing the shared contracts in their own class-library project is recommended.

Sections A and B are marked on the polling client and the server. Section C is marked on the duplex client. Build the polling client first; attempt the duplex client once it works.

A. Client Application functionality [14 Marks]

1. **Sign in:** The client app asks for a user ID and signs in. There is no password. If someone currently signed in is already using that ID, the client shows the reason and lets the user try another. [2 Marks]
2. **Channel list:** The client app shows every channel that currently exists and lets the user join one. The list reflects channels created by other people and updates on its own. A user is in at most one channel at a time and returns to this view on leaving. [2 Marks]
3. **Channel creation:** The user can create a channel by giving it a name. If a channel with that name already exists, the client app says so and the user chooses another. [2 Marks]
4. **Conversation:** Inside a channel the user sends messages and sees messages from every other member appear as they arrive. The user also sees a member list that stays up to date as people join and leave. A user who joins part way through sees the messages sent from that point onward; earlier messages are not shown. [2 Marks]

- 5. Private conversation:** The user can select any other member of the channel and start a private conversation with them. An incoming private message opens its own separate window, apart from the main conversation, which names the sender, keeps the history of that exchange between the two users and allows a reply to be sent from within it. Several private conversations can be open at once. **[2 Marks]**
- 6. File sharing:** The user can choose a text or image file (max 2 MB) and share it with the channel. Shared files appear to every member as clickable entries showing at least the file name and who shared it, and clicking one retrieves the file from the server and opens it. Files of a disallowed type or size are refused with a message the user can read. **[3 Marks]**
- 7. Sign out:** Signing out is available at any time, from either view. The client leaves any channel the user is in, releases the user ID and shuts down cleanly. **[1 Mark]**

Keeping the polling client up to date

The polling client must keep itself current without the user doing anything. Use a background thread that periodically asks the server for changes — new messages, member lists, the channel list, shared files — and updates the interface from what comes back. This is a pull strategy, and it is what Section A expects. Choose your polling interval deliberately: too frequent wastes the server, too slow makes the application feel dead.

If you cannot get background threads working, you may fall back on manual "Refresh" buttons. This is accepted, but requirements 2, 4 and 6 then score at most half marks, because the interface no longer keeps itself current.

None of this applies to the duplex client in Section C, which must have neither a polling thread nor a refresh button.

Interface expectations

The client application has two views with navigation between them. The **channel list** view shows the available channels with controls to create and to join. The **channel conversation** view shows the conversation, a message box, the member list, the shared files and a way to leave. Signing out is available at all times.

B. Server functionality [10 Marks]

- 1. User management:** The server signs users in by ID and keeps IDs unique among those currently signed in, rejecting a duplicate with a reason the client can display. An ID is released the moment its user signs out or disconnects and may then be taken by someone else. **[2 Marks]**
- 2. Channel management:** The server creates channels with unique names, moves users in and out, and keeps each user in at most one channel. It holds the authoritative list of channels and their members **[2 Marks]**.
- 3. Message distribution:** A message sent to a channel reaches every member of that channel at the time it was sent, and nobody else. The server keeps no message history and does not replay past messages to a user who joins later. **[2 Marks]**
- 4. Private messaging:** A private message reaches the one named recipient and nobody else. Sender and recipient must be members of the same channel; the server refuses the message otherwise. **[2 Marks]**

5. File handling: The server receives shared files, stores them, and enforces the permitted types and size limit, refusing anything else with a reason. Each file is served only to members of the channel it was shared into. The file contents must travel to and from the server: passing a local file path between clients does not satisfy this requirement and scores zero for it. **[2 Marks]**

C. Duplex Client: Real-time updates by duplex channel **[8 Marks]**

Build a second client application. It offers the user exactly the same functionality as the polling client in Section A, but it does not ask the server for anything on a timer. Instead it registers a callback with the server when it signs in, and the server invokes that callback whenever something happens the client needs to know about. Use a duplex WCF channel over netTcpBinding.

This is a separate WPF application, not a switch or a setting inside the first one. Both clients must build and run from your submitted solution, and both must work against the same server without the server being restarted or reconfigured between them.

- 1. Duplex contract and callback registration:** A correctly declared duplex service contract with a callback contract, hosted over netTcpBinding. Clients register on sign-in and the server tracks which callback belongs to which user. **[2 Marks]**
- 2. Polling and refresh removed:** Channel list changes, member changes, messages, private messages and shared files all reach the client because the server pushed them. No timer, background loop or refresh button is fetching updates anywhere in the core functionality. Where some updates are pushed and others are still fetched, marks are awarded in proportion. **[2 Marks]**
- 3. Thread safety:** Shared server state is reached by many request threads at once and is correctly synchronised. Callbacks arrive on WCF threads, and updates to the interface are correctly marshalled onto the WPF UI thread, so the client does not throw, freeze or corrupt its display under concurrent updates. **[2 Marks]**
- 4. Disconnection handling:** The server detects a client that closes, crashes or is killed: the user ID is released, the user is removed from their channel, and the remaining members are told. Pushing to a callback that has gone away does not bring down the server or block other users. **[2 Marks]**

Additional Guidelines

- The server listens on a fixed address and port known to the clients. You are developing on a single machine, so localhost with a fixed port is fine.
- This is a client-server system, not peer-to-peer. All traffic passes through the server and clients never connect directly to one another.
- Test with one to five concurrent clients. At least three must run at the demonstration.
- All state is held in server memory. No database is required, and nothing needs to survive a server restart.
- Shared files are limited to .png, .jpg, .jpeg, .gif, .bmp and .txt, up to 2 MB each. Anything else is rejected with a reason the user can read.

Submission guidelines

Submit your assignment electronically via Blackboard before the deadline. Each member of a group must submit the same zip file individually via Blackboard.

- Fill out and sign a declaration of originality. A photo, scan or electronically completed form is fine, provided it is complete and readable. Place it inside your project directory as a .pdf, .jpg or .png.
- Zip your entire project directory, leaving nothing out.
- Submit the zip file to the assignment area on Blackboard.
- Re-download, open and run your submitted work to confirm it has been submitted correctly and is not corrupted.

You may make multiple submissions, but only the newest is marked. The late submission policy in the Unit Outline is strictly enforced. **Do not use WinRar, do not nest zip files, and do not email your submission — Curtin's filters silently discard emails carrying potentially executable attachments.**

Marking demonstration

You must demonstrate and discuss your application with a marker in a one-to-one session, and most of the marks are derived from it. The marker will ask you to rebuild and run your application and to demonstrate its major features and ask you about any aspect of your submission. You will be asked for your contribution percentage, which adjusts your share of the group mark. If you worked individually, your contribution is 100%.

The schedule and policy will be published on Blackboard. If demonstrations are cancelled for unavoidable reasons, marking will be by full inspection.

Run at least three clients. Expect to be asked to sign in with a duplicate user ID, to send public and private messages, to share and open an image and a text file, and to close a client with the window X button while others are still connected. If you have built the duplex client, expect to run both clients side by side against the same server, to send a single message and show it arriving on each of them, and to walk the marker through your callback contract, your locking, and the absence of any timer or refresh path in the duplex client's code.

Academic integrity

This is an assessable task. If you use someone else's work, or obtain someone else's assistance to complete part of the assignment that is intended for you to complete yourself, you will have compromised the assessment. You will not receive marks for any part of your submission that is not your original work.

Further, if you do not reference external sources you are committing plagiarism and collusion, and penalties for Academic Misconduct may apply. Please see Curtin's Academic Integrity website for information on academic misconduct, which includes plagiarism and collusion.

The unit coordinator may require you to provide an oral justification of, or to answer questions about, any piece of written work submitted in this unit. Your responses may be referred to as evidence in an Academic Misconduct inquiry.

End of Assignment 1A